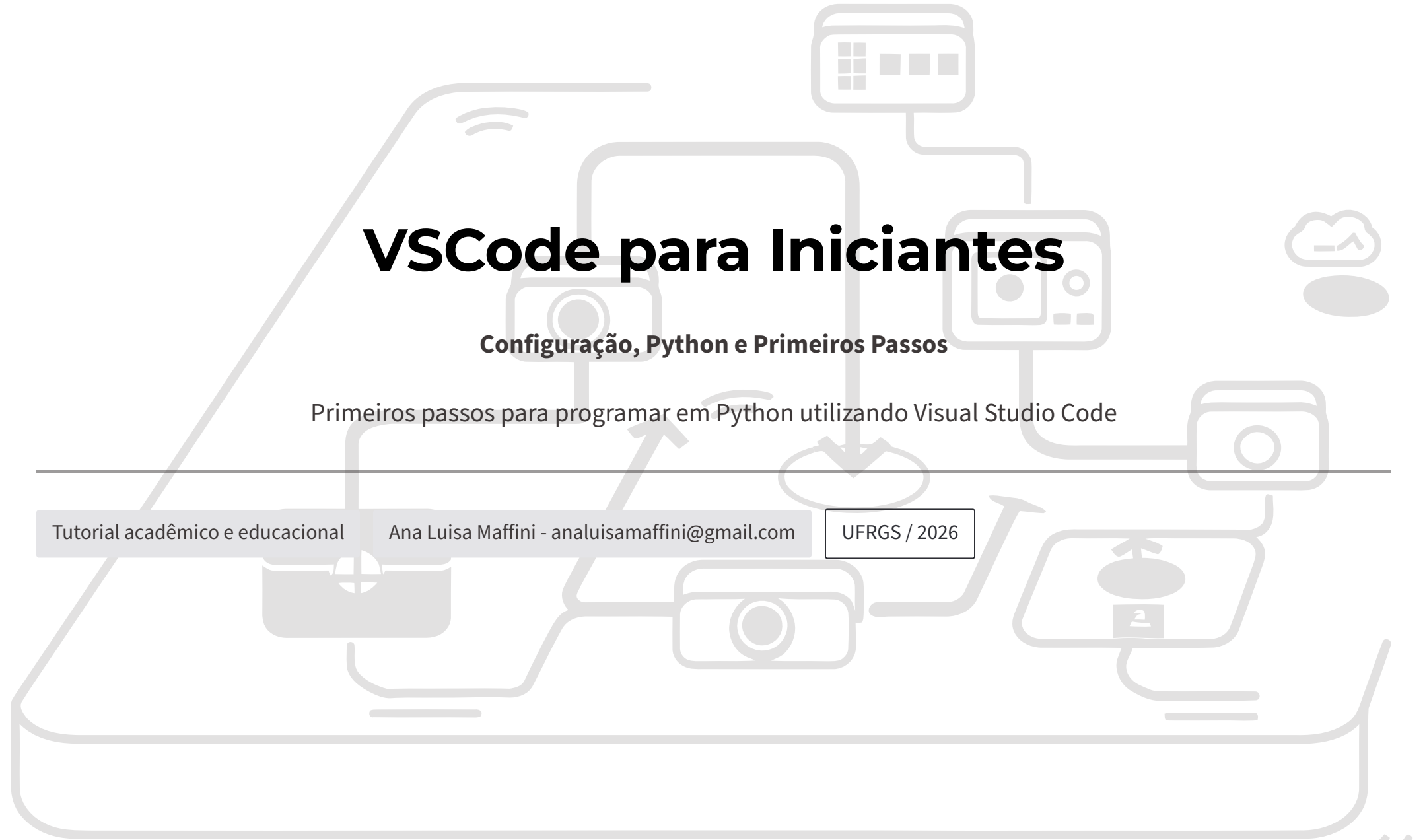




# VSCode para Iniciantes

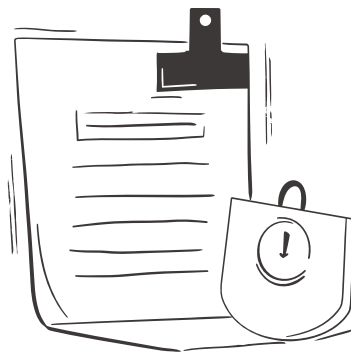
**Configuração, Python e Primeiros Passos**

Primeiros passos para programar em Python utilizando Visual Studio Code

Tutorial acadêmico e educacional

Ana Luisa Maffini - [analuisamaffini@gmail.com](mailto:analuisamaffini@gmail.com)

UFRGS / 2026



# USO DO MATERIAL E DIREITOS AUTORAIS

Este guia de usuário do VSCode, "VSCode para Iniciantes: Configuração, Python e Primeiros Passo", representa o esforço e a dedicação da pesquisadora Ana Luisa Maffini, sendo, portanto, sua propriedade intelectual. O material foi meticulosamente desenvolvido com o propósito de apoiar atividades acadêmicas, científicas e educacionais, visando democratizar o conhecimento sobre programação e seu uso para análises espaciais.

## Uso Permitido

Acreditamos no compartilhamento do conhecimento para o avanço da comunidade. Assim, o uso deste material é **autorizado** para:

- Utilização em atividades de ensino e aprendizado.
- Estudo individual e pesquisa acadêmica.
- Apresentações ou trabalhos científicos, desde que não haja finalidade comercial.

É fundamental que, em todas as utilizações permitidas, a autoria de Ana Luisa Maffini seja devidamente atribuída, respeitando o trabalho intelectual envolvido.

## Uso Proibido

Para garantir a integridade e os direitos da autora, as seguintes ações são **expressamente proibidas**:

- Uso comercial do conteúdo em qualquer formato.
- Venda, redistribuição ou monetização do material.
- Reprodução total ou parcial sem a devida atribuição de créditos.
- Alteração, adaptação ou reutilização para fins comerciais sem autorização prévia por escrito.

**i** O conteúdo apresentado possui caráter exclusivamente educacional e demonstrativo, e a sua compreensão é fundamental para o uso responsável. É importante ressaltar que, devido à evolução constante das tecnologias, este material poderá sofrer atualizações periódicas para refletir novas versões de softwares e metodologias. A versão mais recente sempre buscará oferecer a informação mais precisa e relevante.

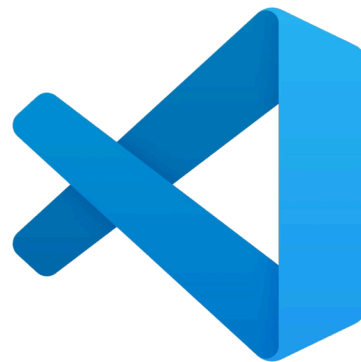
# O que é o VSCode?

O **Visual Studio Code (VSCode)** é um editor de código **gratuito** desenvolvido pela **Microsoft**.

Ele permite:

- ✓ escrever códigos
- ✓ executar scripts
- ✓ instalar extensões
- ✓ trabalhar com notebooks
- ✓ desenvolver projetos de ciência de dados e SIG

«O VSCode é hoje uma das ferramentas mais utilizadas para programação em Python.»



# Por que usar VSCode?

O VSCode é especialmente útil para trabalhos acadêmicos e análises espaciais porque concentra tudo em um único ambiente.



## Python

Suporte nativo à linguagem  
Python com destaque de sintaxe



## Jupyter Notebook

Integração completa com  
notebooks interativos



## Markdown

Permite escrever e visualizar  
documentação formatada



## Organização

Facilita a organização de projetos complexos



## Extensões

Milhares de extensões para ampliar funcionalidades

«Uma única ferramenta pode concentrar códigos, notebooks e documentação.»

# VSCode, Jupyter ou Spyder?

Existem diferentes ambientes para trabalhar com Python. Conheça as diferenças e saiba qual usar em cada situação.

| Ferramenta              | Melhor uso             | Perfil                   |
|-------------------------|------------------------|--------------------------|
| <b>VSCode</b>           | Projetos completos     | Intermediário / Avançado |
| <b>Jupyter Notebook</b> | Análises exploratórias | Ciência de dados         |
| <b>Spyder</b>           | Aprendizado inicial    | Iniciante                |

 Neste tutorial utilizaremos o **VSCode**, pois ele reúne as vantagens dos notebooks e scripts em um único ambiente.

# Conhecendo a interface do VSCode

Ao abrir o VSCode pela primeira vez, você encontrará diferentes áreas da interface. Não se preocupe, cada elemento tem uma função clara.

1

## Barra Lateral

Acesso rápido a arquivos, extensões e controle de versão

2

## Explorador de Arquivos

Visualiza a estrutura de pastas e arquivos do projeto

3

## Área do Editor

Onde você escreve e edita seus códigos

4

## Terminal

Executa comandos diretamente no sistema

5

## Barra de Status

Exibe informações como versão do Python e erros

# Interface do VSCode em ação

## INTERFACE DO VSCODE: CONHEÇA AS PRINCIPAIS ÁREAS

**1 BARRA LATERAL**  
Acesso rápido a arquivos, busca, controle de versão, extensões e configurações. Passe o mouse sobre os ícones para ver a função de cada um.

**2 EXPLORADOR DE ARQUIVOS**  
Mostra a estrutura de pastas e arquivos do projeto. Aqui você pode abrir, criar, mover e organizar seus arquivos.

**3 ÁREA DO EDITOR**  
Onde você escreve, edita e visualiza seus códigos. Aqui podem ser abertas várias abas de arquivos para trabalhar em diferentes códigos ao mesmo tempo.

**4 TERMINAL**  
Executa comandos diretamente no sistema, como instalar bibliotecas, rodar scripts e ferramentas.

**5 BARRA DE STATUS**  
Exibe informações sobre o arquivo aberto, linguagem utilizada, versão do Python, erros e avisos.

```
classificar_overture.py
1 import geopandas as gpd
2 import json
3
4 # Caminho do arquivo de entrada
5 entrada = r"c:\Users\Ana Maffini\Downloads\Dados_Overture_Lajeado.gpkg"
6
7 # Caminho do arquivo de saída
8 saida = r"c:\Users\Ana Maffini\Downloads\Dados_Overture_Lajeado_classificado.gpkg"
9
10 # Nome da camada dentro do GeoPackage
11 camada = "Dados_Overture_Lajeado"
12
13 # Ler o GeoPackage
14 gdf = gpd.read_file(entrada, layer=camada)
15
16 def extrair_taxonomy_texto(valor):
17     """
18     Transforma a coluna taxonomy em texto pesquisável.
19     """
20     if valor is None:
21         return ""
22     try:
23         tax = json.loads(valor) if isinstance(valor, str) else valor
```

PS C:\Projeto\_Overture> python classificar\_overture.py  
Processando dados...  
Arquivo salvo em: C:\Projeto\_Overture\resultados\dados\_classificados.gpkg  
comercio 1245  
servico 2136  
outros 358  
Name: categoria\_atividade, dtype: int64  
PS C:\Projeto\_Overture>

Ln 1, Col 1 Espaços: 4 UTF-8 CRLF Python 3.11.5 64-bit

### RESUMO RÁPIDO

|                                                 |                                                                   |                                              |                                           |                                                   |
|-------------------------------------------------|-------------------------------------------------------------------|----------------------------------------------|-------------------------------------------|---------------------------------------------------|
| <b>1 Barra Lateral</b><br>Atalhos e ferramentas | <b>2 Explorador de Arquivos</b><br>Navegação de pastas e arquivos | <b>3 Área do Editor</b><br>Edição de códigos | <b>4 Terminal</b><br>Execução de comandos | <b>5 Barra de Status</b><br>Informações e alertas |
|-------------------------------------------------|-------------------------------------------------------------------|----------------------------------------------|-------------------------------------------|---------------------------------------------------|

# O que você aprenderá?

Este tutorial cobre todos os fundamentos necessários para começar a programar em Python com VSCode.



## Criar arquivos Python

Aprenda a criar e nomear arquivos .py corretamente



## Organizar projetos

Estruture seus projetos de forma profissional



## Instalar extensões

Configure o ambiente com as extensões essenciais



## Usar o terminal

Execute comandos e instale bibliotecas pelo terminal



## Trabalhar com notebooks

Crie análises interativas com Jupyter Notebook

«Vamos começar preparando nosso ambiente de trabalho.»





# Abrindo o VSCode

Após instalar o programa, abra o Visual Studio Code. A tela inicial apresenta opções para começar rapidamente.

## → Tela inicial

Apresenta atalhos para criar ou abrir projetos recentes

## → Opções recentes

Acesso rápido a pastas e arquivos abertos anteriormente

## → Abertura de pastas

Permite abrir uma pasta existente como projeto



A aparência pode variar conforme o tema instalado no VSCode.

# Trabalhando com pastas de projeto

No VSCode, recomenda-se trabalhar sempre dentro de uma pasta. Isso garante organização e evita erros comuns.

## Exemplo de estrutura

```
Projeto_SIG/  
|— scripts/  
|— dados/  
|— resultados/  
|— notebooks/
```

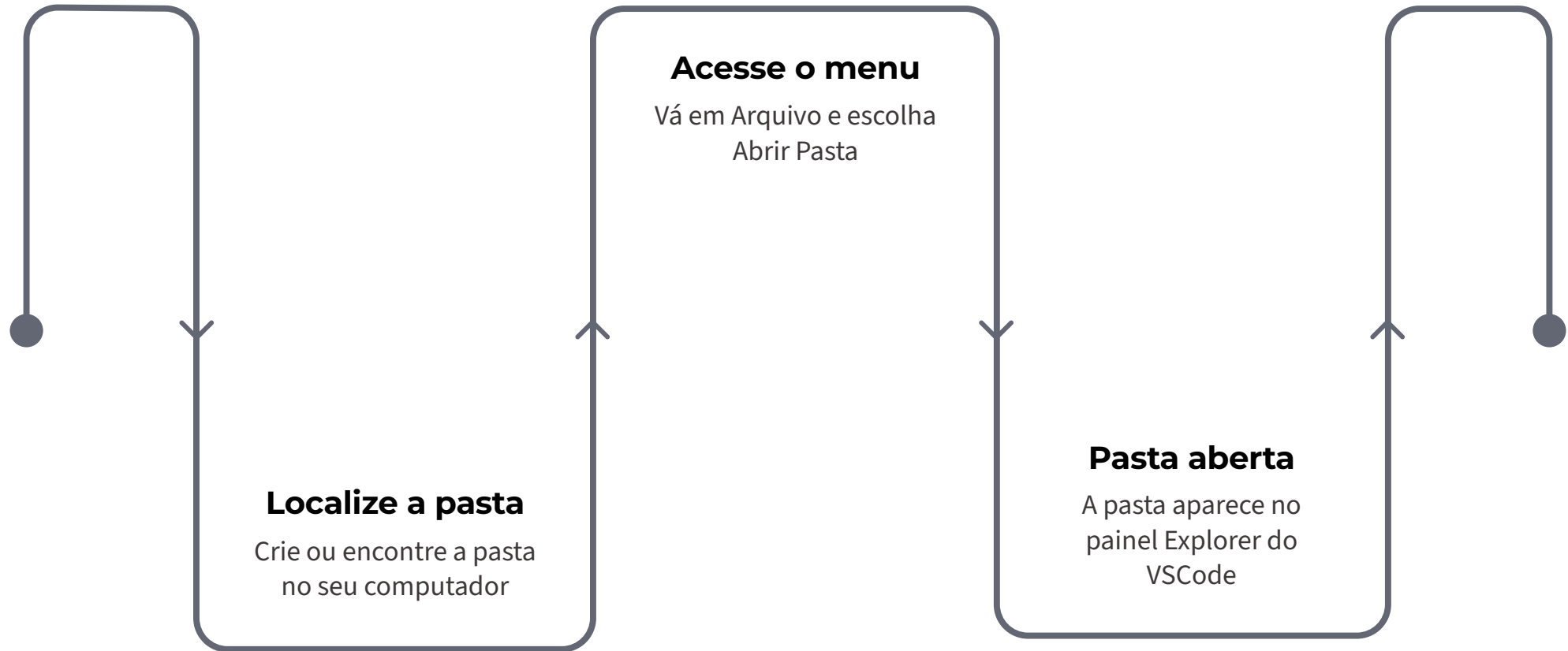
## Benefícios

- ✓ Melhor organização dos arquivos
- ✓ Menos erros de caminho
- ✓ Fácil localização dos arquivos
- ✓ Facilita o trabalho em equipe



Trabalhar com pastas é uma das melhores práticas de programação.

# Como abrir uma pasta no VSCode



Siga esses três passos simples para abrir sua pasta de projeto no VSCode e começar a trabalhar de forma organizada.

 Caminho: **Arquivo → Abrir Pasta** — ou arraste a pasta diretamente para a janela do VSCode.

# Criando um arquivo Python

Após abrir uma pasta, você está pronto para criar seu primeiro arquivo Python.

**1**

## **Clique em Novo Arquivo**

Use o ícone de novo arquivo no explorador de arquivos

**2**

## **Defina um nome**

Escolha um nome descritivo para o arquivo

**3**

## **Salve com extensão .py**

A extensão `.py` identifica o arquivo como Python



Exemplo: `meu_primeiro_codigo.py` — use nomes sem espaços e sem acentos.

# Salvando arquivos

## Caminho pelo menu

Arquivo → Salvar

---

## Atalho de teclado

**Ctrl + S**

## Por que salvar frequentemente?

- Evita perda de trabalho em caso de falha
- O VSCode indica arquivos não salvos com um ponto na aba
- Algumas funcionalidades só funcionam após salvar



Dica prática: salve frequentemente para evitar perda de trabalho!

# .py, .ipynb ou .md?

Cada extensão de arquivo tem um propósito específico no VSCode. Escolher a correta evita confusões.

## **.py**

### **Script Python tradicional**

Use para código que será executado diretamente

Exemplo: analise.py

## **.ipynb**

### **Notebook Jupyter**

Use para experimentos e análises exploratórias

Exemplo: exploracao.ipynb

## **.md**

### **Arquivo Markdown**

Use para documentação e relatórios

Exemplo: README.md



**Regra simples:** Código → `.py` · Experimentos → `.ipynb` · Documentação → `.md`

# O que são extensões?

As extensões adicionam novas funcionalidades ao VSCode. Elas transformam um editor básico em um ambiente completo de desenvolvimento.

Elas permitem:

- ✓ programar em Python com suporte avançado
- ✓ usar notebooks Jupyter diretamente
- ✓ visualizar e editar Markdown
- ✓ melhorar produtividade com atalhos

«Sem extensões, o VSCode é apenas um editor básico. Com elas, torna-se um ambiente completo de ciência de dados.»



# Abrindo a aba de extensões

A aba de extensões é onde você encontra, instala e gerencia todas as extensões do VSCode.

## Pelo ícone

Clique no ícone de **Extensions** na barra lateral esquerda

---

## Pelo atalho

**Ctrl + Shift + X**

## O que você encontrará

- Campo de pesquisa para buscar extensões
- Lista de extensões instaladas
- Extensões recomendadas pelo VSCode
- Botão de instalação e desativação



# Extensões essenciais

Instale as seguintes extensões para preparar seu ambiente Python completo.

## ✓ Python

Suporte oficial à linguagem Python — **obrigatória**

## ✓ Pylance

Autocompletar inteligente e detecção de erros — **obrigatória**

## ✓ Jupyter

Suporte a notebooks .ipynb — **obrigatória**

## ✓ Markdown All in One

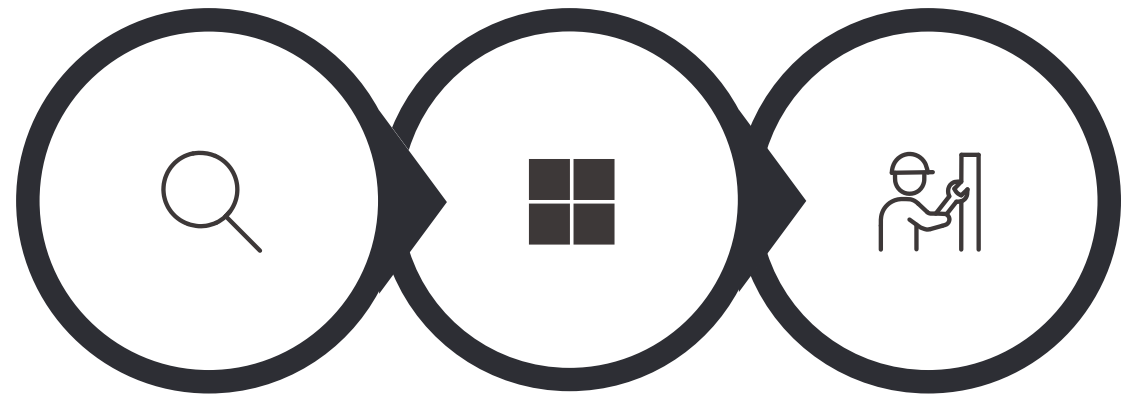
Edição e preview de Markdown — **obrigatória**



Na próxima etapa iremos instalar cada uma delas passo a passo.

# Instalando a extensão Python

A extensão Python é a mais importante para este tutorial. Ela é desenvolvida e mantida oficialmente pela **Microsoft**.



**Procurar  
Python**

**Escolher  
Microsoft**

**Clicar Instalar**

## Suporte à linguagem

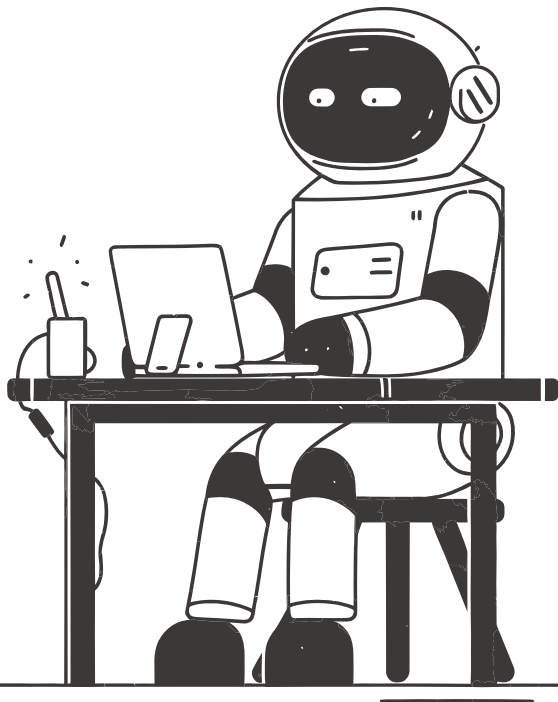
Destaque de sintaxe e reconhecimento de código Python

## Execução de scripts

Permite rodar arquivos .py diretamente no VSCode

## Integração

Conecta o VSCode ao interpretador Python instalado



# Instalando o Pylance

A extensão **Pylance** melhora significativamente a experiência de programação em Python no VSCode.

- ✓ Autocompletar inteligente
- ✓ Detecção de erros em tempo real
- ✓ Sugestões automáticas de código
- ✓ Melhor desempenho geral

✓ Dica: o Pylance geralmente é instalado automaticamente junto com a extensão Python.

## Como instalar

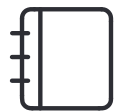
Na aba Extensions, pesquise:

Pylance

Selecione a extensão oficial da Microsoft e clique em **Install**.

# Instalando a extensão Jupyter

A extensão Jupyter transforma o VSCode em um ambiente completo para notebooks interativos.



## Abrir notebooks

Permite abrir e editar arquivos `.ipynb` diretamente no VSCode



## Executar células

Execute células de código individualmente ou todas de uma vez



## Visualizar gráficos

Exibe gráficos e visualizações diretamente no notebook



Essa extensão será muito importante para as análises espaciais deste tutorial.

# Instalando Markdown All in One

## Como instalar

Na aba Extensions, pesquise:

Markdown *All in One*

Instale a extensão e reinicie o VSCode se necessário.

## O que ela oferece

- ✓ Escrever documentação formatada
- ✓ Visualizar preview do Markdown
- ✓ Atalhos rápidos de formatação
- ✓ Melhor organização do projeto



Muito útil para relatórios técnicos e documentação de scripts acadêmicos.

# Conferindo se tudo foi instalado

Após instalar as extensões, verifique se todas estão ativas e funcionando corretamente.



## Acesse a aba Extensions

Use `Ctrl + Shift + X` para abrir  
a aba de extensões



## Verifique o status

Cada extensão instalada deve  
exibir o botão **"Installed"** ou  
**"Disable"**



## Confirme as quatro essenciais

Python ✓ · Pylance ✓ · Jupyter  
✓ · Markdown All in One ✓



Se todas aparecerem como instaladas, seu ambiente está pronto para o próximo passo!

# O que é o Python Interpreter?



O **interpretador Python** é o programa responsável por executar seus códigos. Sem ele, o VSCode não consegue rodar Python.

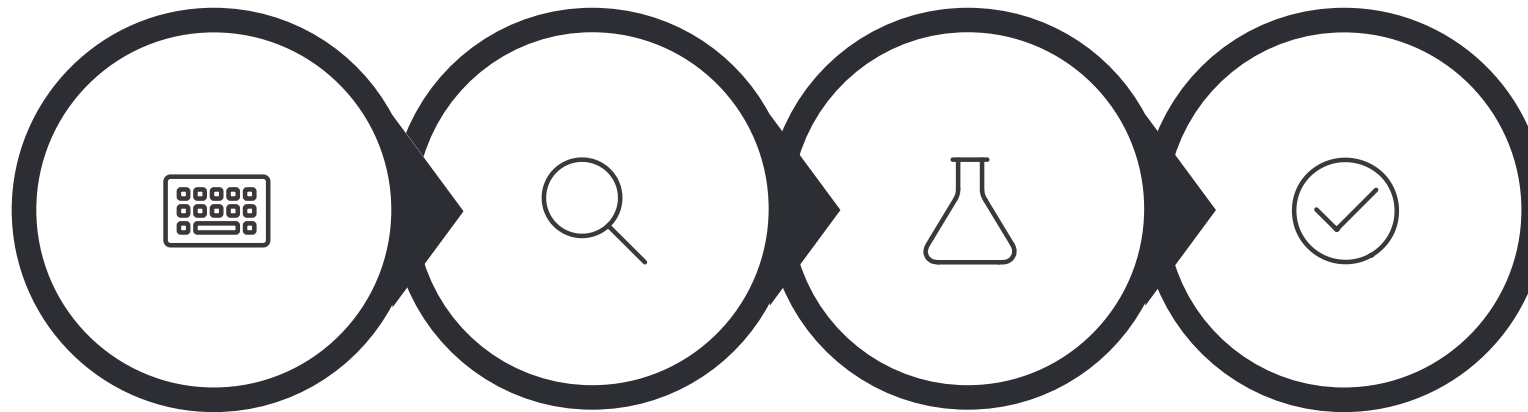
Em muitos computadores existem:

- ▲ Múltiplas versões do Python instaladas
- ▲ Ambientes virtuais diferentes
- ▲ Instalações do Anaconda ou Miniconda

Por isso é fundamental selecionar o interpretador correto.

«Grande parte dos erros de iniciantes vem do Python errado selecionado no VSCode.»

# Selecionando o Python correto



Abrir Paleta

Pesquisar  
Comando

Escolher  
Versão

Confirmar  
Status

## Atalho

# Ctrl + Shift + P

Depois pesquise: Python: Select Interpreter

## Recomendação

Selecione a **versão mais recente** do Python instalada no computador. Prefira versões 3.10 ou superiores para melhor compatibilidade com bibliotecas de SIG.



Algumas bibliotecas Python não funcionam corretamente na versão 3.13 do Python!



# Verificando o Python selecionado

Após selecionar o interpretador, o VSCode exibe a versão ativa na parte inferior da janela.

## Barra inferior do VSCode

Na parte inferior da tela aparecerá algo como: Python 3.12.x

## Sem aviso de erro

Se não houver nenhum ícone de alerta vermelho, o Python está configurado corretamente

## Ambiente ativo

Verifique se o ambiente virtual ou instalação correta está selecionada



Clique na versão exibida na barra inferior para trocar rapidamente o interpretador.

# Testando se o Python está funcionando

## Crie o arquivo

```
teste_python.py
```

## Digite o código

```
print("Olá mundo")
```

## Execute com

```
Run Python File
```

## Resultado esperado

O terminal deverá exibir:

```
Olá mundo
```



Se a mensagem aparecer, parabéns! Seu Python está funcionando corretamente no VSCode.



Se aparecer um erro, verifique se o interpretador Python está selecionado corretamente.

# O que é o terminal?

O terminal é um espaço onde executamos comandos diretamente no computador, sem precisar de interface gráfica.



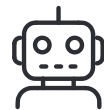
## Rodar scripts

Execute arquivos Python diretamente pelo terminal com um simples comando



## Instalar bibliotecas

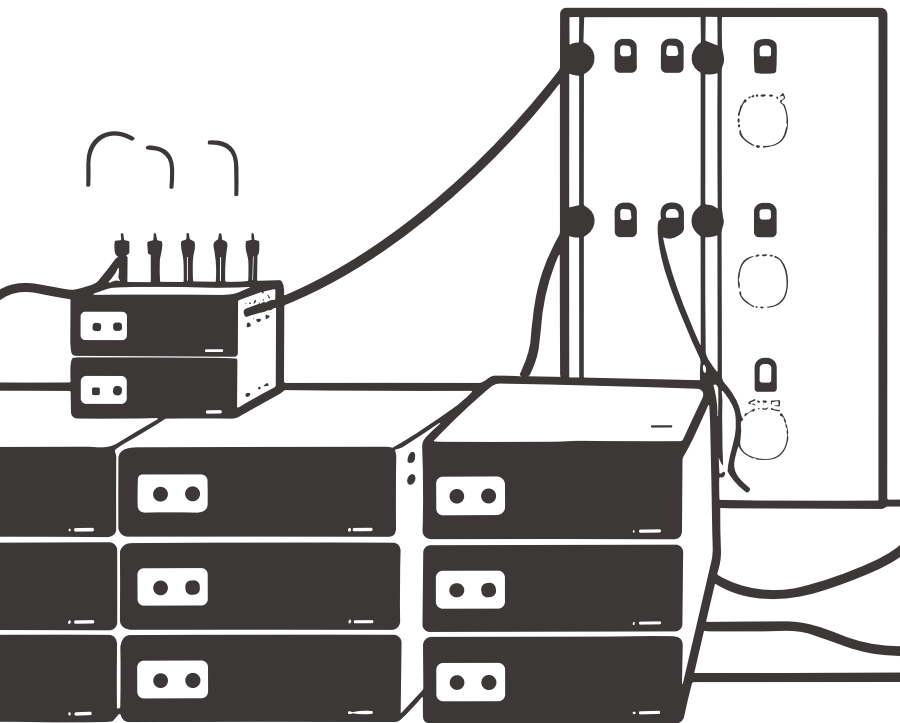
Use o pip para baixar e instalar qualquer biblioteca Python disponível



## Automatizar tarefas

Execute sequências de comandos para automatizar processos repetitivos

«O terminal parece assustador no início, mas será um dos seus maiores aliados na programação.»



# Abrindo o terminal

## Pelo menu

Terminal → New Terminal

---

## Pelo atalho

**Ctrl + `**

## O que você verá

Um painel aparecerá na parte inferior do VSCode com um prompt de comando pronto para receber instruções.

```
PS C:\Projeto_SIG> _
```

 O terminal abre automaticamente na pasta do projeto que está aberto no VSCode.

# Primeiros comandos do terminal

Antes de instalar bibliotecas, verifique se o Python e o pip estão disponíveis no terminal.

## Verificar Python

```
python --version
```

Resultado esperado: Python 3.12.x

## Verificar pip

```
pip --version
```

Resultado esperado: pip 24.x.x

✔ Se ambos os comandos retornarem versões, seu ambiente está pronto para instalar bibliotecas!

✘ Se aparecer "command not found", o Python pode não estar no PATH do sistema. Verifique a instalação.

# Rodando comandos no terminal

## Iniciando o Python interativo

```
python
```

## Resultado esperado

```
Python 3.12.x  
Type "help" for more info.  
>>>
```

## Para sair

```
exit()
```

## Como funciona

Digite um comando e pressione **Enter** para executar. O terminal responde imediatamente.

O símbolo `>>>` indica que o Python está aguardando um comando.



O modo interativo é ótimo para testar pequenos trechos de código rapidamente.

# Próximo passo: bibliotecas Python

Agora que o ambiente está funcionando, vamos aprender a instalar as ferramentas que tornam o Python poderoso para análises espaciais.

**1**

## **Instalar bibliotecas**

Aprenda a usar o pip para baixar ferramentas prontas

**2**

## **Utilizar o pip**

Domine o gerenciador de pacotes do Python

**3**

## **Preparar para SIG**

Configure Python para análises espaciais e geoprocessamento

«Sem bibliotecas, Python possui recursos limitados para análise espacial. Com elas, torna-se uma ferramenta poderosa de SIG.»

# O que são bibliotecas Python?

Bibliotecas são **conjuntos de códigos prontos** criados para facilitar tarefas específicas. Em vez de programar tudo do zero, reutilizamos soluções existentes.

Exemplos de uso:

- ✓ Análise de dados tabulares
- ✓ Criação de mapas interativos
- ✓ Geração de gráficos
- ✓ Geoprocessamento vetorial e raster
- ✓ Estatísticas e modelagem

«Grande parte do poder do Python vem de suas bibliotecas. Elas são o coração do ecossistema científico.»





# O que é o pip?

O **pip** é o sistema responsável por instalar bibliotecas Python. Com ele, baixamos ferramentas prontas diretamente da internet.



## Exemplo de uso

```
pip install pandas
```

## Em outras palavras

**pip = instalador de bibliotecas Python**

Assim como uma loja de aplicativos instala apps no celular, o pip instala bibliotecas no Python.

# Instalando uma biblioteca

## Comando no terminal

```
pip install pandas
```

Depois pressione **Enter**

---

## Resultado esperado

```
Successfully installed pandas-2.x.x
```

## O que acontece?

01

---

### Download automático

O pip baixa a biblioteca do repositório PyPI

02

---

### Instalação

A biblioteca é instalada no ambiente Python ativo

03

---

### Confirmação

O terminal exibe "Successfully installed"

# Testando se a instalação funcionou

**Crie um arquivo Python e digite:**

```
import pandas as pd  
print("Biblioteca instalada com sucesso!")
```

**Resultado esperado:**

Biblioteca instalada com sucesso!



**Atenção — Erro comum:**

Se aparecer:

```
ModuleNotFoundError: No module named  
'pandas'
```

Provavelmente o Python selecionado no VSCode está incorreto. Verifique o interpretador ativo na barra inferior.



Se o código rodar sem erros, a biblioteca está instalada e funcionando corretamente!

# Bibliotecas essenciais para análises espaciais

Para análises urbanas e espaciais, algumas bibliotecas são especialmente importantes. Juntas, elas transformam Python em uma poderosa ferramenta de SIG.

## **Pandas**

Tabelas e dados tabulares

## **GeoPandas**

Dados geoespaciais vetoriais

## **Shapely**

Geometrias espaciais

## **PyProj**

Projeções e CRS

## **Rasterio**

Dados raster

## **OSMnx**

Redes OpenStreetMap

## **Contextily**

Mapas base

## **Matplotlib**

Gráficos e visualizações

# Pandas

O **Pandas** é a biblioteca mais utilizada para trabalhar com tabelas e dados tabulares em Python.

## Principais usos:

- ✓ Leitura de arquivos CSV e Excel
- ✓ Limpeza e transformação de dados
- ✓ Estatísticas descritivas
- ✓ Organização e filtragem de tabelas
- ✓ Exportação de resultados

## Como importar

```
import pandas as pd
```

## Exemplo básico

```
df = pd.read_csv("dados.csv")  
print(df.head())
```

# GeoPandas

O **GeoPandas** adiciona capacidades espaciais ao Pandas, permitindo trabalhar com dados geográficos vetoriais diretamente em Python.

## Formatos suportados

- Shapefiles (.shp)
- GeoPackage (.gpkg)
- GeoJSON (.geojson)

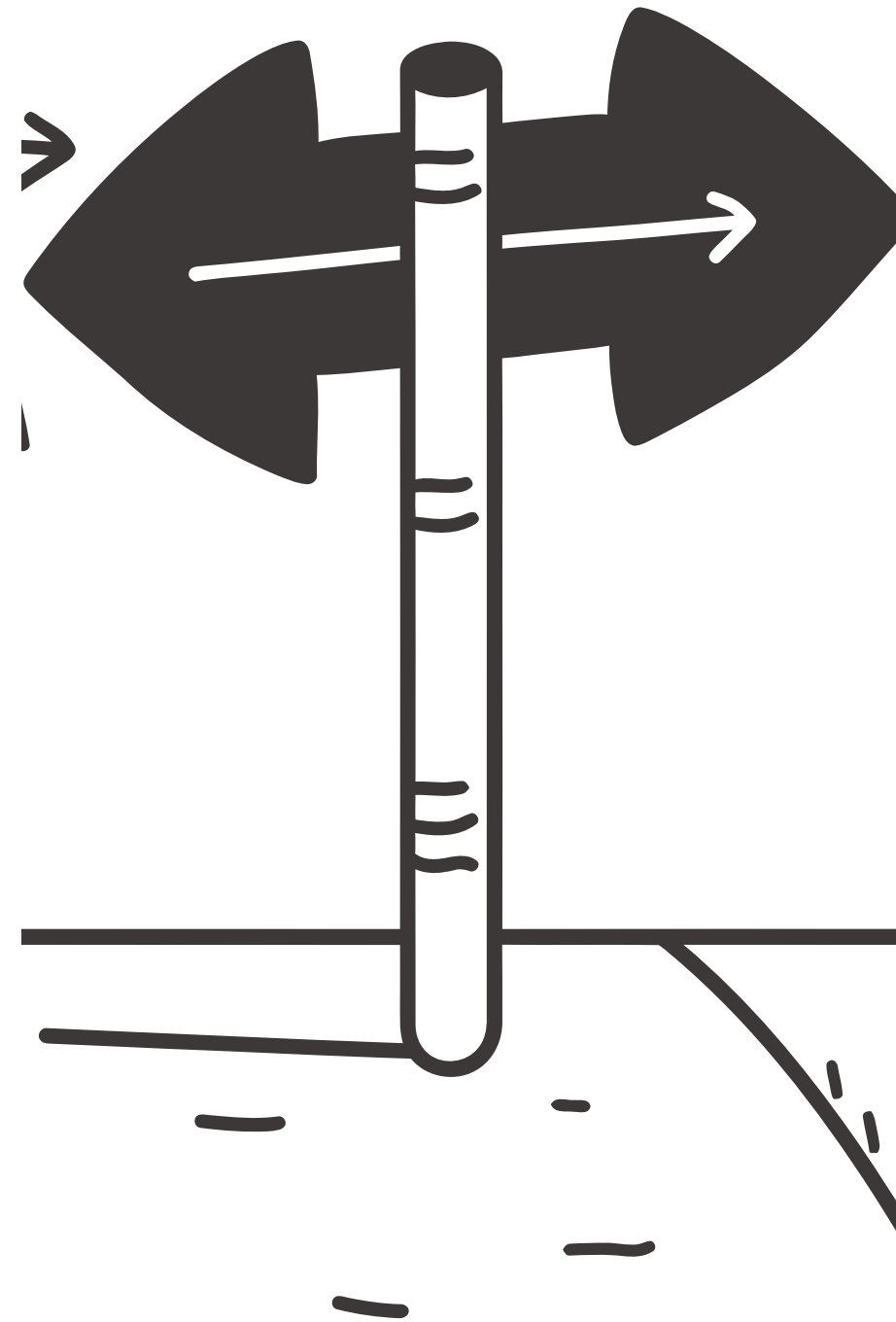
## Análises espaciais

- Interseções e uniões
- Buffers e dissolve
- Junções espaciais

## Como importar

```
import geopandas as gpd
```

```
gdf = gpd.read_file("mapa.shp")
```



# Shapely e PyProj

## Shapely

Manipulação de geometrias espaciais

- ✓ Criação de buffers
- ✓ Cálculo de interseções
- ✓ Medição de áreas
- ✓ Cálculo de distâncias

```
from shapely.geometry import Point
ponto = Point(0, 0)
```

## PyProj

Conversão entre sistemas de coordenadas

- ✓ Reprojeções de dados
- ✓ Definição de CRS
- ✓ Trabalho com códigos EPSG
- ✓ Transformações geodésicas

```
from pyproj import CRS
crs = CRS("EPSG:4326")
```

# Outras bibliotecas importantes

Estas bibliotecas são especialmente úteis para análises urbanas, de mobilidade e acessibilidade territorial.



## Rasterio

Trabalha com dados raster como imagens de satélite, modelos digitais de elevação e grades de dados contínuos



## OSMnx

Baixa e analisa redes de ruas e infraestrutura urbana diretamente do OpenStreetMap



## Contextily

Adiciona mapas base (tiles) como fundo para visualizações geoespaciais criadas com GeoPandas



Muito úteis para: urbanismo · mobilidade · acessibilidade · análise territorial



# Instalando bibliotecas essenciais

Você pode instalar várias bibliotecas ao mesmo tempo com um único comando no terminal.



A instalação pode demorar alguns minutos dependendo da sua conexão com a internet.

```
pip install geopandas pandas shapely pyproj rasterio osmnx contextily folium matplotlib numpy jupyter notebook
```

## O que acontece durante a instalação

O pip baixa cada biblioteca e suas dependências automaticamente do repositório PyPI.

## Após a instalação

Verifique se todas foram instaladas com:

```
pip list
```

# Conferindo bibliotecas instaladas

## Comando

```
pip list
```

## Resultado esperado

```
Package Version
-----
geopandas 0.14.x
pandas    2.x.x
shapely    2.x.x
matplotlib 3.x.x
...
```

## Para que serve?

O comando `pip list` exibe todas as bibliotecas instaladas no ambiente Python ativo.



Use essa verificação sempre que estiver em dúvida sobre quais bibliotecas estão disponíveis.



Para verificar uma biblioteca específica: `pip show pandas`

# Rodando um script Python

Após salvar um arquivo `.py`, existem duas formas principais de executá-lo no VSCode.

## Método 1 — Botão Run

Clique no botão ► **Run Python File** no canto superior direito do editor

Ideal para: iniciantes e execuções rápidas

## Método 2 — Terminal

Digite `python nome_do_arquivo.py` no terminal integrado

Ideal para: projetos maiores e uso profissional



Ambos os métodos produzem o mesmo resultado. Escolha o que for mais confortável para você.

# Método 1 — Run Python File

## Passo a passo

1. Abra o arquivo `.py` no editor
2. Localize o botão ► **Run Python File** no canto superior direito
3. Clique no botão
4. O resultado aparece no terminal inferior

## Vantagens

- ✓ Muito simples e visual
- ✓ Não requer conhecimento do terminal
- ✓ Ideal para iniciantes

- ✓ O terminal abrirá automaticamente na parte inferior do VSCode para exibir o resultado.

# Método 2 — Rodando pelo terminal

## Sintaxe do comando

```
python nome_do_arquivo.py
```

## Exemplo prático

```
python teste_python.py
```

## Depois pressione

```
Enter
```

## Por que usar o terminal?

- ✓ Método utilizado em projetos maiores
- ✓ Permite passar argumentos ao script
- ✓ Mais controle sobre a execução
- ✓ Padrão em ambientes profissionais



Dica: use a tecla **Tab** para autocompletar o nome do arquivo no terminal.

# Próximo passo: Jupyter Notebook

Agora que você sabe rodar scripts Python, vamos aprender a trabalhar com notebooks interativos, uma das ferramentas mais poderosas para ciência de dados e SIG.

1

## Criar notebooks

Aprenda a criar arquivos `.ipynb` no VSCode

2

## Executar células

Execute código bloco a bloco de forma interativa

3

## Análises exploratórias

Misture texto, código, gráficos e tabelas em um único documento

«Jupyter Notebook é uma das ferramentas mais utilizadas em ciência de dados e SIG.»

# O que é o Jupyter Notebook?

O Jupyter Notebook é um ambiente interativo que permite misturar código, texto, gráficos e tabelas em um único documento.

## Código Python

Execute células de código e veja os resultados imediatamente abaixo

## Texto e explicações

Documente sua análise com texto formatado em Markdown

## Gráficos e tabelas

Visualize resultados diretamente no notebook, sem abrir janelas externas



Muito utilizado em: ciência de dados · análises espaciais · pesquisas acadêmicas · experimentação

«Um notebook funciona como um "caderno digital de análise", onde código e explicação coexistem.»

# Criando um notebook Jupyter

Existem duas formas de criar um novo notebook Jupyter no VSCode.

## Método 1 — Novo arquivo

Crie um novo arquivo com a extensão:

```
.ipynb
```

Exemplo: `analise_espacial.ipynb`

## Método 2 — Paleta de comandos

Use o atalho:

```
Ctrl + Shift + P
```

Pesquise:

Create New Jupyter Notebook



O notebook abrirá automaticamente com uma célula de código pronta para uso.



# Como funciona um notebook?

Um notebook é dividido em **células**. Cada célula pode conter código ou texto, e pode ser executada individualmente.

## Célula de Código

Executa Python e exibe o resultado logo abaixo da célula

```
import pandas as pd
df = pd.DataFrame({"A": [1,2,3]})
df
```

## Célula Markdown

Escreve texto formatado, títulos, listas e explicações metodológicas

```
# Análise Espacial
Esta seção apresenta os resultados...
```

-  Vantagem: é possível alternar explicações e códigos no mesmo documento, criando uma narrativa analítica completa.

# Primeiro código no notebook

**Digite em uma célula de código:**

```
print("Olá notebook!")
```

**Depois clique em:**

Run Cell

**Resultado esperado:**

Olá notebook!

**O que acontece?**

01

---

**Célula executada**

O kernel Python processa o código da célula

02

---

**Resultado exibido**

O output aparece imediatamente abaixo da célula

03

---

**Número de execução**

Um número entre colchetes indica a ordem de execução

# Como executar células

Existem diferentes formas de rodar uma célula no Jupyter Notebook do VSCode.



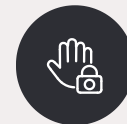
## Botão Run Cell

Clique no ícone ► à esquerda da célula para executá-la



## Shift + Enter

Executa a célula atual e avança para a próxima automaticamente



## Ctrl + Enter

Executa a célula atual sem avançar para a próxima



Dica prática: execute uma célula por vez enquanto aprende. Isso facilita identificar erros.

# Rodando o notebook inteiro

## Como fazer

Quando todas as células estiverem prontas, clique em:

Run All

Localizado na barra superior do notebook.

## Muito útil para:

- ✓ Reproduzir análises completas
- ✓ Verificar se há erros em alguma célula
- ✓ Atualizar todos os resultados de uma vez
- ✓ Compartilhar análises reproduzíveis



Atenção: erros em uma célula interrompem a execução das células seguintes.

# Selecionando o Kernel correto

No notebook é necessário escolher o **kernel Python**, o ambiente que executará o código.

## O que é o kernel?

O kernel corresponde ao ambiente Python que executará o notebook. Ele deve ter todas as bibliotecas necessárias instaladas.

## Como selecionar

Clique no seletor de kernel no canto superior direito do notebook e escolha a versão correta do Python.

## O que verificar

Versão correta do Python ·  
Bibliotecas instaladas · Ambiente virtual ativo



Erro comum: abrir o notebook com o Python errado, fazendo com que as bibliotecas não sejam encontradas.

# O que é Markdown?

Markdown é uma linguagem simples usada para formatar textos de forma rápida e legível, sem precisar de editores complexos.

## Documentação

Escreva documentação técnica clara e bem formatada

## README

Crie arquivos README.md para descrever seus projetos

## Relatórios técnicos

Produza relatórios acadêmicos diretamente no VSCode

## Notebooks

Documente análises dentro de células Markdown nos notebooks

«Markdown permite documentar suas análises de forma profissional e reproduzível.»



# Criando um arquivo Markdown

## Extensão do arquivo

.md

## Exemplo mais comum

README.md

## Outros exemplos

relatorio.md  
metodologia.md  
anotacoes.md

## Para que serve o README.md?

- ✓ Documentação do projeto
- ✓ Instruções de uso
- ✓ Anotações e observações
- ✓ Relatórios técnicos



O arquivo README.md é exibido automaticamente em plataformas como GitHub, tornando seu projeto mais profissional.

# Escrevendo títulos em Markdown

Os títulos em Markdown são criados com o símbolo #. Quanto mais #, menor o título.

## Código Markdown

```
# Título principal  
## Subtítulo  
### Seção menor  
#### Seção ainda menor
```

## Resultado formatado

# Título principal

## Subtítulo

### Seção menor

#### Seção ainda menor



Use títulos para organizar seu documento em seções claras e navegáveis.



# Listas e destaques

Markdown oferece formatação simples para listas, negrito, itálico e outros elementos textuais.

## Código Markdown

- item 1
- item 2
- item 3

**\*\*texto em negrito\*\***

*\*texto em itálico\**

``código inline``

## Resultado formatado

- item 1
- item 2
- item 3

**texto em negrito**

*texto em itálico*

`código inline`

# Visualizando o Markdown

## Pelo menu

Clique em **Open Preview** no canto superior direito do editor

---

## Pelo atalho

# Ctrl + Shift + V

## Tela dividida

Você pode visualizar o código Markdown e o preview formatado lado a lado:

1. Abra o arquivo `.md`
2. Use `Ctrl + Shift + V`
3. Arranje as abas lado a lado



A visualização atualiza em tempo real conforme você edita o arquivo Markdown.

# Organização de projetos

Uma estrutura organizada evita problemas futuros e facilita o trabalho colaborativo e a reprodutibilidade das análises.

## Estrutura recomendada

```
Projeto_SIG/  
|— dados/  
|— scripts/  
|— notebooks/  
|— resultados/  
|— docs/  
|— README.md
```

## Por que organizar assim?



### **dados**

Shapefiles, CSVs e  
arquivos de entrada



### **scripts**

Arquivos .py com  
código reutilizável



### **notebooks**

Análises  
exploratórias em  
.ipynb



### **resultados**

Mapas, gráficos e  
outputs gerados



Dica: evite deixar tudo na mesma pasta. A organização economiza horas de trabalho no futuro.

# Erros comuns de iniciantes

Conhecer os erros mais frequentes ajuda a resolvê-los rapidamente e a não se frustrar durante o aprendizado.

## △ Python errado selecionado

Verifique sempre o interpretador ativo na barra inferior do VSCode

## △ Biblioteca não instalada

Use `pip install nome_biblioteca` e confirme com `pip list`

## △ Arquivo salvo sem extensão

Sempre salve com `.py`, `.ipynb` ou `.md` conforme o tipo

## △ Terminal fechado acidentalmente

Reabra com `Ctrl + `` ou pelo menu Terminal → New Terminal

## △ Erro de caminho da pasta

Sempre abra o VSCode dentro da pasta do projeto para evitar erros de caminho

«Errar faz parte do processo de aprendizado. Cada erro é uma oportunidade de entender melhor como o sistema funciona.»

# Você agora sabe usar VSCode para Python!



Neste tutorial você percorreu todos os fundamentos necessários para programar em Python com VSCode.

## ✓ Projetos

Organizar pastas e arquivos de forma profissional

## ✓ Extensões

Instalar e configurar Python, Pylance, Jupyter e Markdown

## ✓ Python

Selecionar o interpretador correto e testar o ambiente

## ✓ Terminal

Usar comandos e instalar bibliotecas com pip

## ✓ Scripts

Rodar arquivos .py pelo botão Run e pelo terminal

## ✓ Notebooks

Criar e executar análises interativas com Jupyter

## ✓ Markdown

Documentar projetos com arquivos .md formatados

## ✓ SIG

Instalar bibliotecas espaciais como GeoPandas e OSMnx

«Programar é uma habilidade construída pela prática. Continue explorando, errando e aprendendo.»

# Parabéns! Você chegou ao fim desse tutorial.

## CONCLUSÃO

Agora é hora de começar a praticar, é com a experiência prática que as habilidades realmente ganham forma. A melhor maneira de aprender é fazendo: abrindo o software, carregando dados, cometendo erros, explorando possibilidades e testando caminhos diferentes. Cada projeto traz um aprendizado novo, e cada desafio ajuda você a evoluir com mais confiança.

## Em Caso de Dúvidas e Para Mais Informações Entre em Contato



[analuisamaffini@gmail.com](mailto:analuisamaffini@gmail.com)



[analuisamaffini@ufrgs.br](mailto:analuisamaffini@ufrgs.br)